

Assessment Task Template



Assessment title: **Conceptual Design and Debugging**

Assessment method: Portfolio

Unit information

Course name: **22603VIC Certificate IV in Cyber Security**

Unit[s] of competency: **CTPRG434 Automate Processes**

Assessment information for student

Task number: 2 of 3

Due date: See Gordon Online

Where assessment will take place: The Gordon, in-class

Time available to complete this assessment: Refer to due date on Gordon Online

Assessment requirements

This assessment is comprised of a series of tasks that cover aspects of conceptual design, such as flow-charts, pseudocode, and hand-tracing, and basic debugging. You must complete the tasks and submit a Word document or PDF containing your solutions to each task.

Specific assessment requirements (if any)

N/A

The Performance and Knowledge skills you must demonstrate

This assessment will test your ability to:

- Develop algorithmic solutions to presented problems
- Debug simple programs

Your assessor will provide these resources

- Unit resources and materials
- Computer and internet access if necessary, and access to Gordon Online.
- Access to drawing/modelling software

You (the learner) will need to provide these resources

Assessment Task Template



- Personal computer
- Storage device (USB drive/SSD)

Plagiarism

The Gordon maintains its training integrity and has processes in place to identify plagiarism. Where a student uses material from other authors, the original source must be referenced. Penalties apply for substantiated cases of plagiarism; penalties may include recording a Not Satisfactory result for the assessment and may result in a Fail being recorded against the specific unit linked to the plagiarism.

Requirements of this assessment task

Carefully develop solutions to the presented tasks and compile your answers in a Word document or PDF.

If you are comfortable using markdown, a good suggestion is to use an editor like MarkText or Typora, which support code blocks and flowcharts via Mermaid syntax.

How the task is to be submitted

Via Gordon online

How feedback will be provided

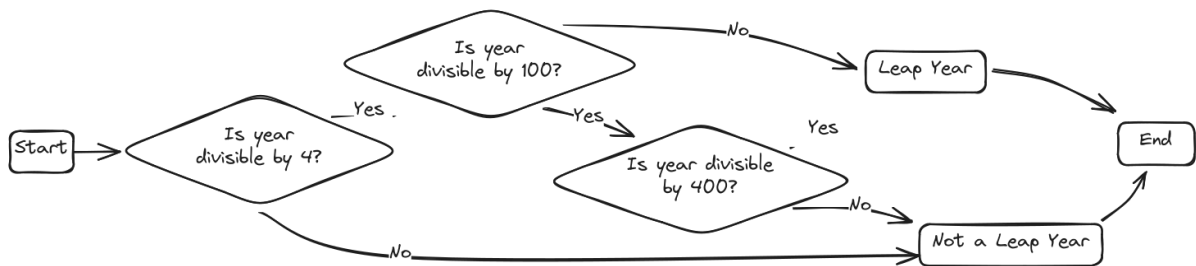
Feedback will be provided via Gordon online

Assessment Task Template



Flow Charts

Analyse the following problems and develop algorithms to solve them. For each problem, create a flow chart that describes your solution, ensuring that you take **all possible outcomes** and inputs into account. You can use a tool such as <https://www.tldraw.com/> to create this if you wish, or the integrated Mermaid flow-charts in MarkText and Typora. For example:



Please note that **you are not writing any code for these**, only drawing the flowcharts.

1. The Collatz Conjecture

The Collatz Conjecture, also known as the " $3n + 1$ problem", is a classic problem in mathematics and computer science that is simple to understand but remains unsolved. The conjecture asks whether repeating two simple arithmetic operations will eventually transform every positive integer into 1. The rules are as follows:

- Start with any positive integer n .
- If n is even, divide it by 2.
- If n is odd, multiply it by 3 and add 1.
- Continue this process until n becomes 1.

However, whether this number is **certain** to ever become 1 is a mystery in mathematics.

An example of the number sequences we can expect might be $n = 12$, for which we get the sequence: 12, 6, 3, 10, 5, 16, 8, 4, 2, 1. In comparison, if we use $n = 27$, there are 111 steps before we eventually drop to 1.

Your task is to draw a flowchart for a program that asks the user for a number, then applies the Collatz Conjecture to that number. It should keep a total of the number of steps in the sequence, as well as print out the actual sequence.

Example output:

```

Enter a positive integer: 11
The Collatz sequence for 11 is: 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1
Total number of steps in the sequence: 15
  
```

Assessment Task Template

OFFICIAL



2. Reverse Guess the Number Game

In the Reverse Guess the Number game, the computer takes on the role of the guesser, and the human player thinks of a number within a defined range. The objective for this task is to create a flowchart that outlines the logical steps the computer should take to efficiently guess the number using a [binary search algorithm](#).

1. The player is prompted to think of a number within a specified range (e.g., between 1 and 1000).
2. The computer attempts to guess the number.
3. After each guess, the player informs the computer whether the guess was too high, too low, or correct.
4. The computer uses the feedback to adjust its next guess, narrowing down the possible range based on binary search principles.
5. The process continues until the computer guesses the correct number.

Requirements:

- Implement a binary search approach where the computer's guess is always the middle of the current possible range.
- Ensure that the player's responses are valid (i.e., 'high', 'low', or 'correct'). If the response is invalid, prompt for a valid response.
- Based on the player's feedback, adjust the range of possible numbers for the next guess.
- Track and display the number of guesses made by the computer.
- After the correct number is guessed, ask the player if they want to play again. Restart the game if yes, and end the program if no.

Example output:

```
Think of a number between 1 and 1000 and I will guess it.
Is your number 500? (high, low, correct): high
Is your number 250? (high, low, correct): low
Is your number 375? (high, low, correct): correct
I guessed your number in 3 tries!
Play again? (y/n): n
Goodbye!
```

Assessment Task Template



Hand-Tracing

3. Multiplication Table

If we were to design an algorithm to print a 5x5 multiplication table in Python, it would look something like this:

```
n = 5 # The number of rows and columns

for i in range(1, n + 1):
    for j in range(1, n + 1):
        print(i * j, end="\t")
    print("\n")
```

Create a trace table for a 4 x 4 multiplication table, showing the values of i , j and the output ($i * j$) at each step. The table should resemble the following:

Iteration	i	j	Output
1			
2			
3			
4			
5			
...			

Fill in the blanks, and add rows and columns as required.

4. The Fibonacci Sequence

The Fibonacci sequence is a series of numbers in which each number (after the first two) is the sum of the two preceding it. This sequence is one of the most famous formulas in mathematics, often found in various aspects of art, nature, and mathematics. The mathematical formula for the sequence is $F(n) = F(n - 1) + F(n - 2)$. In Python, the function might look like this:

```
def fibonacci(n):
    a, b = 0, 1
    for i in range(n):
        print(a)
        a, b = b, a + b
```

Create a trace table for the above function as you did with the previous exercise, showing the values of a , b , and the output of the program. The value of n should be at least 5.

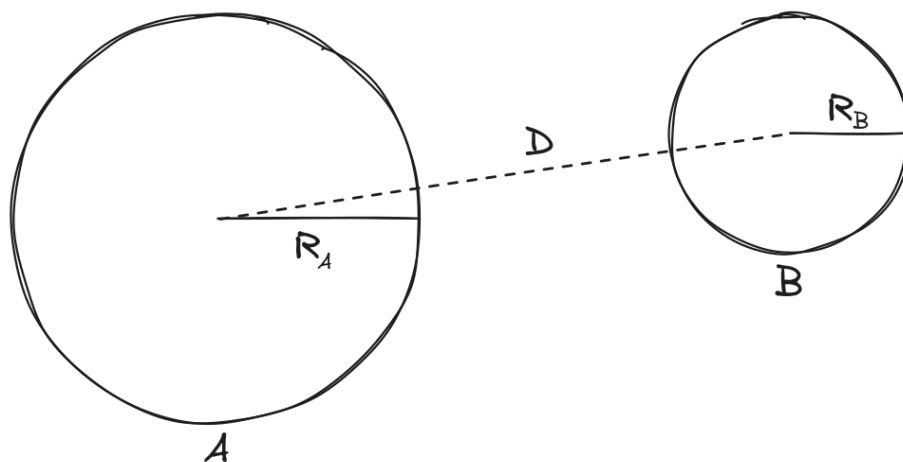
Assessment Task Template



Pseudocode

5. Relationship Between Circles

Design a program that determines the relationship between two circles based on their radii and the distance between their centres. The pseudocode should include steps for obtaining input, performing calculations, and determining if the circles overlap, if one is inside the other, or if they are separate.



Given the radius of **Circle A** (R_A), the radius of **Circle B** (R_B), and the **distance** between them (D), we can determine:

- Circles are overlapping: $(R_A + R_B) > D > |R_A - R_B|$
- Circle A is inside Circle B: $(D + R_A) < R_B$
- Circle B is inside Circle A: $(D + R_B) < R_A$
- Circles do not overlap:
 - $D > (R_A + R_B)$ or:
 - $D < |R_A - R_B|$

Your program must ask the user for the radius of each circle, then ask for the distance between them. It will then calculate whether or not the circles are overlapping, and if so, calculate whether one circle is inside the other (and which one). As with previous exercises, user input must be checked to ensure input is valid and converted from strings to integers.

6. Bubble Sort

[Sorting algorithms](#) are fundamental in the field of computer science and programming, playing a crucial role in organising and managing data efficiently. There are many different of sorting algorithms, such as the [selection sort](#), [insertion sort](#), [merge sort](#), [quick sort](#), [bucket sort](#), etc. Each sorting algorithm has its own unique characteristics, efficiencies, and use

Assessment Task Template



cases. The simplest sorting algorithm, both in terms of understanding and implementation, is typically considered to be the [bubble sort](#).

Here's a basic overview of how Bubble Sort works:

- Start at the beginning of the array or list.
- Compare the first two elements.
- If the first element is greater than the second, swap them.
- Move to the next pair of elements (second and third) and repeat the comparison and swap if necessary.
- Continue this process for each pair of adjacent elements until the end of the list. This completes a single pass through the list.
- Repeat the entire process for $n - 1$ passes (where n is the number of elements in the list), or until no more swaps are needed (which means the list is sorted).

Your task is to create pseudocode that implements the bubble sort algorithm.

7. Walking Directories

Sometimes the best way to solve an algorithm is to implement what is known as [recursion](#). Recursion is a concept in programming where a function calls itself in order to solve a problem, each of which can be solved by applying the same process. It's a powerful technique often used to simplify complex problems. A basic recursive function (in Python) looks like this:

```
def recursive_function(n: int):
    print(f"Recursive call {n}!")
    recursive_function(n + 1)

recursive_function(1)
```

Which will print:

```
Recursive call 1!
Recursive call 2!
Recursive call 3!
Recursive call 4!
Recursive call 5!
...
Continues until stack overflow!
```

Your task is to create a **recursive function** that lists all files in a given directory. The user must be prompted to enter a directory path, and the program should descend into all subdirectories, printing the full path of each file it finds. You may create any helper functions you wish, or you can assume they exist.

Assessment Task Template



Debugging

This task involves examining programs written in Bash, PowerShell, or Python, and identifying and correct any errors present. These errors could range from syntax and logic mistakes to more subtle issues like incorrect algorithm implementation or improper handling of edge cases. To answer this question, you can either provide the entire corrected code or a description of the error.

8. URL Checker Script

Alice, an IT professional, decides to use the following Bash script to check the connectivity of a list of URLs. She runs the script without passing any arguments and expects to see a usage message, but the script doesn't provide any feedback and seems to do nothing.

```
#!/usr/bin/env bash

# This script reads a list of URLs from the command line,
# checks them for connectivity and logs the results to a file.

log_file="connectivity.log"

# Check connectivity to a URL
check_connectivity() {
    local url=$1
    if wget --spider --timeout=5 -q "$url"; then
        echo "$url: Accessible" >>"$log_file"
    else
        echo "$url: Inaccessible" >>"$log_file"
    fi
}

# Main
if [ $# -lt 0 ]; then
    echo "Usage: $0 <url1> <url2> ..."
    exit 1
fi

for url in "$@"; do
    check_connectivity "$url"
done
```

9. Running Host Script

Bob, a network administrator, tries to use this script to check the connectivity of several servers. While the script appears to run normally, some of the hosts he knows are running appear to be down.

Assessment Task Template



```
#!/usr/bin/env bash

# Ping hosts and print whether they are up or down
ping_hosts() {
    for host in "$@"; do
        if ping -c 1 -W 1 "$host" &>/dev/null; then
            echo "Connection to $host failed"
        else
            echo "Connection to $host successful"
        fi
    done
}

# If no hosts are passed to the script, print usage message and exit
if [ $# -eq 0 ]; then
    echo "Usage: $0 <host1> <host2> ..."
    exit 1
else
    ping_hosts "$@"
fi
```

10. File Sorter

Jordan, a software developer, uses the following script to sort a list of user names stored in a file. However, when he runs the script, the entries in the output file are in the same order as the input file. The entries don't appear to be sorted at all!

```
#!/usr/bin/env python3
# coding: utf-8

# Python program to sort users from a file, and write the sorted
# users to another file.

import sys

def read_users_from_file(file_path: str) -> list[str]:
    """ Read a list of users from a file """
    with open(file_path, 'r') as file:
        users = file.readlines()
    return users

def write_users_to_file(users: list[str], file_path: str):
    """ Write a list of users to file """
    with open(file_path, 'w') as file:
```

Assessment Task Template



```
        for user in users:
            file.write(user)

def sort_users(users: list[str]) -> list[str]:
    """ Sort users by name using bubble sort """
    for i in range(len(users) - 1):
        for j in range(i + 1, len(users)):
            if users[i] > users[j]:
                users[i], users[j] = users[j], users[i]
    return users

def main():
    args = sys.argv[1:]
    if len(args) != 2:
        print("Usage: python sort_names.py <input_file> <output_file>")
        sys.exit(1)

    input_file = args[0]
    output_file = args[1]

    users = read_users_from_file(input_file)
    sorted_users = sort_users(users)
    write_users_to_file(sorted_users, output_file)

if __name__ == "__main__":
    sys.exit(main())
```

Assessment Task Template



Assessment criteria

The list below contains the criteria against which you will be assessed, for this assessment. You must satisfactorily address every criterion listed to pass the assessment.

Criteria

1. Ability to develop algorithmic solutions to presented tasks
2. Ability to perform basic visual debugging of script code
3. Understanding of basic principles of algorithm design